

cuGMEC Parameter Table

This document corresponds to `src/cuGMEC_param.h` . You must recompile after changing these parameters.

The Example column shows only one possible way to write each parameter; it does not imply a recommended value.

Notation:

Notation	Meaning
<Species>	Ion , Alpha , Beam
<MHDField>	Phi , A , dNe , dTe
<PICField>	dPi , dPa , dPb

Data Types and File Paths

Parameter	Allowed Values	Description	Notes	Example
mhdReal	float / double	MHD calculation precision.	MHD precision should be greater than or equal to PIC precision.	using mhdReal = double;
picReal	float / double	PIC calculation precision.	PIC precision should be less than or equal to MHD precision.	using picReal = float;
inputDir	String path	Input file directory.	For a run from scratch, only the MHDEquilibrium file is needed. For a continued run, the MHDContinue and PICContinue files are also needed. They come from the previous run's outputDir/final .	const std::string inputDir = "/path/to/input";
outputDir	String path	Output file directory.		const std::string outputDir = "/path/to/output";
MHDEquilibrium	.bin file name	MHD equilibrium file.	Generated by generateInput2D.m for tokamaks and generateInput3D.m for stellarators.	const std::string MHDEquilibrium = "MHDEquilibrium_384_32.bin";
<Species>PhaseSpaceMapping	.bin file name	Phase-space mapping file.	Required only when ifOutputPhaseSpace* is enabled; generated by generatePhaseSpaceMapping2D.m .	const std::string IonPhaseSpaceMapping = "IonPhaseSpaceMapping.bin";

Normalization

Parameter	Allowed Values	Description	Notes	Example
B0	Real number	Magnetic-field normalization scale.		const double B0 = 4.921751144619735;
L0	Real number	Length normalization scale.		const double L0 = 6.595629295925759;
VA0	Real number	Velocity normalization scale.		const double VA0 = 8.864164667700194e+06;
RH00	Real number	Left endpoint of the radial interval.		const double RH00 = 0.08;
RH01	Real number	Right endpoint of the radial interval.		const double RH01 = 0.90;
PSITMAX	Real number	Outermost toroidal magnetic flux (Wb/rad).		const double PSITMAX = 18.868213504765762;

Grid and Parallel Setup

Parameter	Allowed Values	Description	Notes	Example
hostNums	Integer	Number of MPI processes.		const int hostNums = 4;
devNums	Integer	Number of GPUs used by each MPI process.		const int devNums = 4;
gridNx	Integer	Number of radial grid points.		const int gridNx = 256;
gridNy	Integer	Number of field-aligned grid points.	Must be divisible by hostNums * devNums .	const int gridNy = 32;
gridNz	Integer	Number of toroidal grid points.		const int gridNz = 96;
NFP	Integer	Number of field periods. NFP=1 for tokamak; NFP!=1 for stellarator.		const int NFP = 1;
ppcNums	Integer	Average number of particles per species in each grid cell.		const int ppcNums = 256;

Toroidal Range and Initial Perturbation

Parameter	Allowed Values	Description	Notes	Example
tubes	Integer	Simulate $1/\text{tubes}$ of the toroidal domain.		<code>const int tubes = 6;</code>
leftN	Integer	Lower bound of the retained toroidal mode number in the $1/\text{tubes}$ toroidal domain.		<code>const int leftN = 1;</code>
rightN	Integer	Upper bound of the retained toroidal mode number in the $1/\text{tubes}$ toroidal domain.	Retained toroidal modes are <code>tubes*[leftN,rightN]</code> , spaced by <code>tubes</code> .	<code>const int rightN = 6;</code>
refinedTimes	Integer	Poloidal-grid refinement factor for <code>selectNM_*</code> filtering.	The refined poloidal grid size is <code>gridNy * refinedTimes</code> .	<code>const int refinedTimes = 32;</code>
perturbLeftN	Integer	Lower bound of the initial perturbation toroidal mode number in the $1/\text{tubes}$ toroidal domain.		<code>const int perturbLeftN = 1;</code>
perturbRightN	Integer	Upper bound of the initial perturbation toroidal mode number in the $1/\text{tubes}$ toroidal domain.	Initial perturbation modes are <code>tubes*[perturbLeftN,perturbRightN]</code> , spaced by <code>tubes</code> .	<code>const int perturbRightN = 6;</code>
perturbRadialIndex	Integer	Radial peak grid point of the initial Gaussian perturbation.		<code>const int perturbRadialIndex = 85;</code>
perturbWidth	Real number	Radial width of the initial Gaussian perturbation.		<code>const mhdReal perturbWidth = 0.04;</code>
perturbAmplitude	Real number	Normalized <code>Phi</code> amplitude of the initial Gaussian perturbation.	The perturbation is proportional to <code>amplitude * exp[-(X - Xc)^2 / (2 * width^2)]</code> , where <code>Xc = (radialIndex - 1) / (gridNx - 1)</code> .	<code>const mhdReal perturbAmplitude = 2.5e-9;</code>

Filtering

Parameter	Allowed Values	Description	Notes	Example
ifFilterN<MHDField>	<code>trueType</code> / <code>falseType</code>	Whether to apply toroidal filtering to the MHD field.		<code>using ifFilterN_Phi = trueType;</code>
ifFilterN_dP	<code>trueType</code> / <code>falseType</code>	Whether to apply toroidal filtering to the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>using ifFilterN_dP = trueType;</code>
removeN<MHDField>	<code>{}</code> or list of toroidal mode numbers	Remove the specified toroidal mode numbers from the MHD field.		<code>constexpr std::array<int, 1> removeN_Phi = {7};</code>
removeN_dP	<code>{}</code> or list of toroidal mode numbers	Remove the specified toroidal mode numbers from the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>constexpr std::array<int, 0> removeN_dP = {};</code>
selectNM<MHDField>	<code>{{N, leftM, rightM}, ...}</code>	Apply poloidal filtering to the specified toroidal mode numbers of the MHD field.		<code>constexpr std::array<std::tuple<int, int, int>, 1> selectNM_Phi = {{0, 0, 0}};</code>
selectNM_dP	<code>{{N, leftM, rightM}, ...}</code>	Apply poloidal filtering to the specified toroidal mode numbers of the PIC perturbed pressure.	Applies to <code>dPi/dPa/dPb</code> .	<code>constexpr std::array<std::tuple<int, int, int>, 1> selectNM_dP = {{0, 0, 1}};</code>

MHD Physics Parameters

Parameter	Allowed Values	Description	Notes	Example
ifNonlinearMHD	<code>trueType</code> / <code>falseType</code>	Whether to include MHD nonlinear terms.		<code>using ifNonlinearMHD = trueType;</code>
ifEparallel	<code>trueType</code> / <code>falseType</code>	Whether to include the parallel electric-field term.		<code>using ifEparallel = trueType;</code>
ifFLRMHD	<code>trueType</code> / <code>falseType</code>	Whether to include the FLR effect from background thermal ions in the Poisson equation.		<code>using ifFLRMHD = falseType;</code>
ifQNeutrality	<code>trueType</code> / <code>falseType</code>	Whether to compute the electron density perturbation from quasi-neutrality.	When enabled, <code>dNe</code> is determined by vorticity and the ion density perturbation.	<code>using ifQNeutrality = trueType;</code>
ifReducedMHD	<code>trueType</code> / <code>falseType</code>	Evolve the pressure of the background thermal plasma with a single-fluid pressure equation; disables evolution of <code>dNe</code> and <code>dTe</code> .	Use with <code>ifEparallel=falseType</code> .	<code>using ifReducedMHD = falseType;</code>
Gamma	Real number	Adiabatic index in the single-fluid pressure equation.		<code>const mhdReal Gamma = 0.0;</code>
ifMaxwellStress	<code>trueType</code> / <code>falseType</code>	Whether to include Maxwell stress.	Effective only when <code>ifNonlinearMHD=trueType</code> .	<code>using ifMaxwellStress = trueType;</code>
ifReynoldsStress	<code>trueType</code> / <code>falseType</code>	Whether to include Reynolds stress.	Effective only when <code>ifNonlinearMHD=trueType</code> .	<code>using ifReynoldsStress = trueType;</code>
MaxwellStressCoef	Real number	Maxwell stress coefficient.	Effective only when <code>ifMaxwellStress=trueType</code> .	<code>const mhdReal MaxwellStressCoef = 1.0;</code>
ReynoldsStressCoef	Real number	Reynolds stress coefficient.	Effective only when <code>ifReynoldsStress=trueType</code> .	<code>const mhdReal ReynoldsStressCoef = 1.0;</code>

Dissipation

Parameter	Allowed Values	Description	Notes	Example
ifNablaPerp2<MHDField>	<code>trueType</code> / <code>falseType</code>	Enable second-order perpendicular MHD-field dissipation.		<code>using ifNablaPerp2Phi = trueType;</code>
perp2<MHDField>	Real number	Coefficient for second-order perpendicular MHD-field dissipation.		<code>const mhdReal perp2Phi = 1.0e-7;</code>
ifNablaPara4<MHDField>	<code>trueType</code> / <code>falseType</code>	Enable fourth-order parallel MHD-field dissipation.		<code>using ifNablaPara4Phi = trueType;</code>
para4<MHDField>	Real number	Coefficient for fourth-order parallel MHD-field dissipation.		<code>const mhdReal para4Phi = 1.0e-6;</code>
ifNablaPerp2dP	<code>trueType</code> / <code>falseType</code>	Enable second-order perpendicular PIC-pressure dissipation.	Applies to <code>dPi/dPa/dPb</code> .	<code>using ifNablaPerp2dP = trueType;</code>
perp2dP	Real number	Coefficient for second-order perpendicular PIC-pressure dissipation.	Applies to <code>dPi/dPa/dPb</code> .	<code>const mhdReal perp2dP = 1.0e-6;</code>
ifNablaPara4dP	<code>trueType</code> / <code>falseType</code>	Enable fourth-order parallel PIC-pressure dissipation.	Applies to <code>dPi/dPa/dPb</code> .	<code>using ifNablaPara4dP = falseType;</code>
para4dP	Real number	Coefficient for fourth-order parallel PIC-pressure dissipation.	Applies to <code>dPi/dPa/dPb</code> .	<code>const mhdReal para4dP = 0.0;</code>

PIC Physics Parameters

Parameter	Allowed Values	Description	Notes	Example
<code>ifFLRPIC</code>	<code>trueType</code> / <code>falseType</code>	Whether to enable the FLR effect in the PIC component.		<code>using ifFLRPIC = trueType;</code>
<code>ifNonlinearPIC</code>	<code>trueType</code> / <code>falseType</code>	Whether to enable PIC nonlinear terms.		<code>using ifNonlinearPIC = trueType;</code>
<code>gyroNums</code>	Integer	Number of gyro-average points.	Must satisfy <code>gyroNums = 2*n</code> with <code>n >= 2</code> .	<code>const int gyroNums = 4;</code>

PIC Species Parameters

Parameter	Allowed Values	Description	Notes	Example
<code>if<Species></code>	<code>trueType</code> / <code>falseType</code>	Whether to enable the corresponding ion species.		<code>using ifIon = trueType;</code>
<code><Species>Type</code>	<code>Maxwell</code> / <code>Slowing0</code> / <code>Slowing1</code> / <code>Slowing2</code> / <code>Slowing3</code>	Velocity distribution type.		<code>const disType IonType = Maxwell;</code>
<code><Species>Space</code>	<code>spaceReal</code> / <code>spaceUniform</code>	Marker-position sampling: physical spatial distribution or uniform.		<code>const spaceType IonSpace = spaceReal;</code>
<code><Species>Velocity</code>	<code>velocityReal</code> / <code>velocityUniform</code>	Marker-velocity sampling: physical velocity distribution or uniform.		<code>const velocityType IonVelocity = velocityUniform;</code>
<code><Species>Mass</code>	Real number	Ion mass, normalized by the proton mass.		<code>const picReal IonMass = 2.5;</code>
<code><Species>Char</code>	Real number	Ion charge, normalized by the electron charge.		<code>const picReal IonChar = 1.0;</code>
<code><Species>Beta</code>	Real number	On-axis ion beta ($P/(B_0^2/(2\mu_0))$).		<code>const picReal IonBeta = 0.037793721898356;</code>
<code><Species>Vmin</code>	Real number	Lower bound of the ion velocity, normalized by <code>VA0</code> .		<code>const picReal IonVmin = 0.0135;</code>
<code><Species>Vmax</code>	Real number	Upper bound of the ion velocity, normalized by <code>VA0</code> .		<code>const picReal IonVmax = 0.54;</code>
<code><Species>Vb</code>	Real number	Cutoff velocity of the slowing-down distribution.		<code>const picReal BeamVb = 0.1;</code>
<code><Species>DeltaV</code>	Real number	Cutoff width of the slowing-down distribution.		<code>const picReal BeamDeltaV = 0.1;</code>
<code><Species>Lambda0</code>	Real number	Center of <code>Lambda</code> in the anisotropic distribution.		<code>const picReal BeamLambda0 = 0.4;</code>
<code><Species>DeltaLambda2</code>	Real number	Square of the <code>Lambda</code> width in the anisotropic distribution.		<code>const picReal BeamDeltaLambda2 = 1.0 / (4.5 * 4.5);</code>
Distribution Type		Formula		
<code>Maxwell</code>		$f \sim n * T^{(-3/2)} * \exp[-m*V^2/(2*T)]$		
<code>Slowing0</code>		$f \sim n / (V^3 + Vc^3)$		
<code>Slowing1</code>		$f \sim n * [1 + \operatorname{erf}((Vb - V)/\Delta V)] / (V^3 + Vc^3)$		
<code>Slowing2</code>		$f \sim n * \exp[-(\Lambda - \Lambda_{da0})^2 / \Delta \Lambda_{da2}] / (V^3 + Vc^3)$		
<code>Slowing3</code>		$f \sim n * [1 + \operatorname{erf}((Vb - V)/\Delta V)] * \exp[-(\Lambda - \Lambda_{da0})^2 / \Delta \Lambda_{da2}] / (V^3 + Vc^3)$		

Diagnostics

Parameter	Allowed Values	Description	Notes	Example
<code>ifDiagAmplitude</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose mode amplitude.		<code>using ifDiagAmplitude = trueType;</code>
<code>ifDiagFrequency</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose mode frequency.		<code>using ifDiagFrequency = trueType;</code>
<code>ifDiagEparallel</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose the parallel electric field.		<code>using ifDiagEparallel = trueType;</code>
<code>ifDiagDensity</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose the ion density perturbation.		<code>using ifDiagDensity = trueType;</code>
<code>ifDiagDiffusivity</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose ion diffusivity.		<code>using ifDiagDiffusivity = trueType;</code>
<code>ifDiagZFDrive</code>	<code>trueType</code> / <code>falseType</code>	Whether to diagnose the zonal-flow drive source.		<code>using ifDiagZFDrive = falseType;</code>
<code>ifCheckNaN</code>	<code>trueType</code> / <code>falseType</code>	Whether to check for NaN.	If NaN is diagnosed, the program stops immediately and writes output.	<code>using ifCheckNaN = trueType;</code>

MHD Field Output

Parameter	Allowed Values	Description	Notes	Example
ifOutputPhi	trueType / falseType	Whether to output Phi at multiple time points.		using ifOutputPhi = trueType;
ifOutputA	trueType / falseType	Whether to output A at multiple time points.		using ifOutputA = falseType;
ifOutputdNe	trueType / falseType	Whether to output dNe at multiple time points.		using ifOutputdNe = falseType;
ifOutputdTe	trueType / falseType	Whether to output dTe at multiple time points.		using ifOutputdTe = falseType;
ifOutputdP	trueType / falseType	Whether to output the single-fluid pressure dP at multiple time points.		using ifOutputdP = falseType;
ifOutputdPi	trueType / falseType	Whether to output dPi at multiple time points.		using ifOutputdPi = falseType;
ifOutputdPa	trueType / falseType	Whether to output dPa at multiple time points.		using ifOutputdPa = falseType;
ifOutputdPb	trueType / falseType	Whether to output dPb at multiple time points.		using ifOutputdPb = falseType;

Phase-space Output

Parameter	Allowed Values	Description	Notes	Example
gridE	Integer	Number of phase-space grid points in the E direction.	Must match the settings in <code>generatePhaseSpaceMapping2D.m</code> .	const int gridE = 96;
gridPphi	Integer	Number of phase-space grid points in the Pphi direction.	Must match the settings in <code>generatePhaseSpaceMapping2D.m</code> .	const int gridPphi = 128;
gridLambda	Integer	Number of phase-space grid points in the Lambda direction.	Must match the settings in <code>generatePhaseSpaceMapping2D.m</code> .	const int gridLambda = 48;
ppcPhase	Integer	Average number of particles per species in each phase-space grid cell.		const int ppcPhase = 2048;
ifOutputPhaseSpaceJacobian	trueType / falseType	Whether to output the phase-space Jacobian.		using ifOutputPhaseSpaceJacobian = trueType;
ifOutputPhaseSpaceOrbit	trueType / falseType	Whether to output phase-space orbit frequencies.		using ifOutputPhaseSpaceOrbit = falseType;
ifOutputPhaseSpaceF0	trueType / falseType	Whether to output the phase-space equilibrium distribution function.		using ifOutputPhaseSpaceF0 = falseType;
ifOutputPhaseSpaceDeltaF	trueType / falseType	Whether to output the phase-space perturbed distribution function.		using ifOutputPhaseSpaceDeltaF = falseType;
ifOutputPhaseSpacePower	trueType / falseType	Whether to output phase-space wave-particle interaction power.		using ifOutputPhaseSpacePower = falseType;
gridVpara	Integer	Number of pitch-space grid points in the vpara direction.		const int gridVpara = 128;
gridVperp	Integer	Number of pitch-space grid points in the vperp direction.		const int gridVperp = 64;
ppcPitch	Integer	Average number of particles per species in each pitch-space grid cell.		const int ppcPitch = 2048;
ifOutputPitchSpaceJacobian	trueType / falseType	Whether to output the pitch-space Jacobian.		using ifOutputPitchSpaceJacobian = trueType;
ifOutputPitchSpaceF0	trueType / falseType	Whether to output the pitch-space equilibrium distribution function.		using ifOutputPitchSpaceF0 = falseType;
ifOutputPitchSpaceDeltaF	trueType / falseType	Whether to output the pitch-space perturbed distribution function.		using ifOutputPitchSpaceDeltaF = falseType;
ifOutputPitchSpacePower	trueType / falseType	Whether to output pitch-space wave-particle interaction power.		using ifOutputPitchSpacePower = falseType;

Time Advancement

Parameter	Allowed Values	Description	Notes	Example
ifContinue	trueType / falseType	Whether to start from continue files.	The <code>inputDir</code> directory must contain the <code>MHDContinue</code> and <code>PICContinue</code> continue files. They come from the previous run's <code>outputDir/final</code> .	using ifContinue = falseType;
continueSteps	Integer	Number of steps already completed before resuming.	Must match the suffix of the continue file names.	const int continueSteps = 0;
dt	Real number	MHD time step.	Normalized by the Alfvén time <code>L0/VA0</code> .	const double dt = 0.02;
totalSteps	Integer	Total number of MHD steps in this run.		const int totalSteps = 20000;
ratioDt	Integer	Ratio of the PIC time step to the MHD time step.	PIC uses <code>dt * ratioDt</code> for each push.	const int ratioDt = 1;
sortSteps	Integer	Particle sorting interval.		const int sortSteps = 25;
diagSteps	Integer	Diagnostic sampling interval.		const int diagSteps = 1;
outputSteps	Integer	Field and phase-space output interval.		const int outputSteps = 2500;